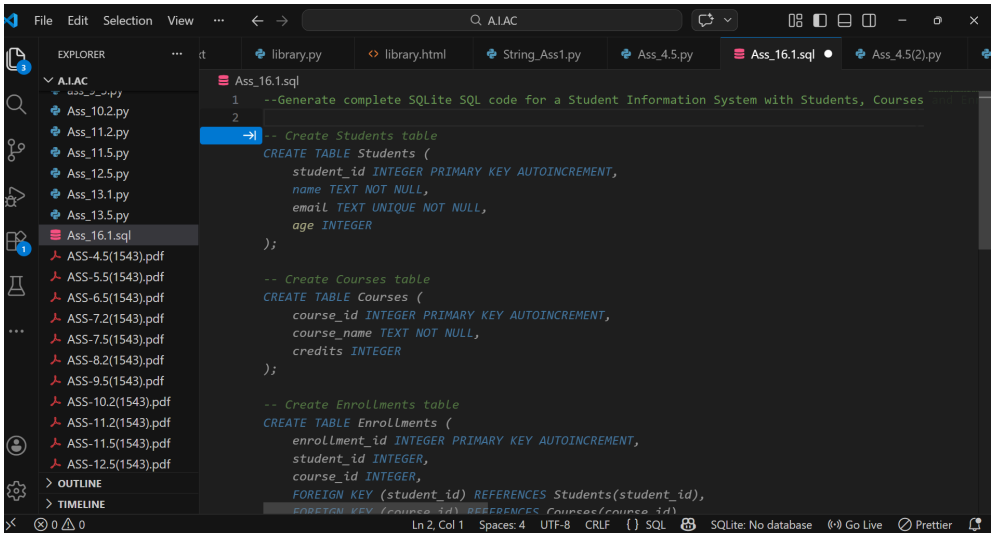
AI Assisted Coding Assignment-16.1

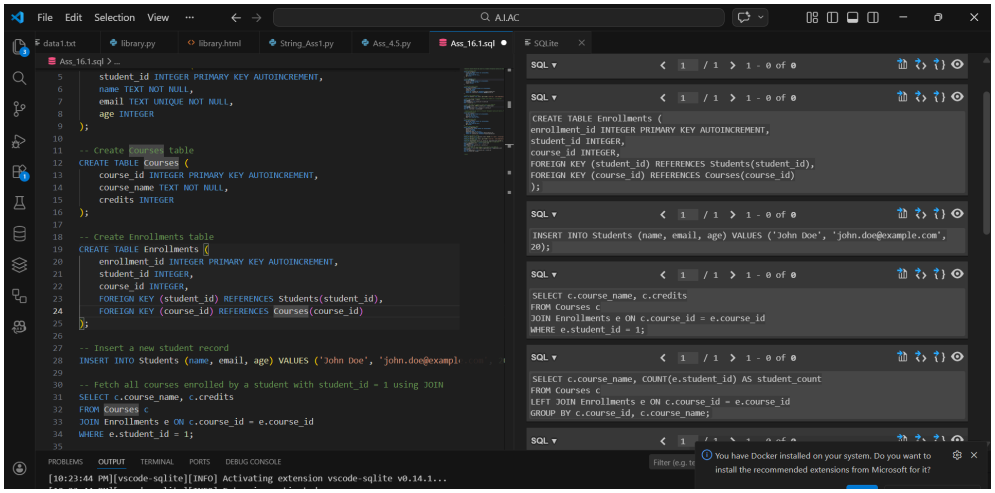
> 2303A51543 BT:29 VELDI.HRUTHIKA 24-3-26

💡 Task 1 – Student Information System Schema
Task:
Design a database schema for a Student Information System and generate queries using AI.
Instructions:
• Tables: Students, Courses, Enrollments.
• Define primary keys, foreign keys, and relationships.
• Generate queries:
o Insert a new student record.
o Fetch all courses enrolled by a student.
o Count number of students in each course.
Expected Output:
• SQL CREATE TABLE statements and queries for student–course relationships.

```
--2303a51543
--Task-1
--Generate complete SQLite SQL code for a Student Information System with Students, Cours
es and Enrollments tables. Include primary keys, foreign keys, insert a student record, f
etch all courses by student_id=1 using JOIN, and count students per course using GROUP BY

-- Create Students table
CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    age INTEGER
);

-- Create Courses table
CREATE TABLE Courses (
    course_id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_name TEXT NOT NULL,
    credits INTEGER
);

-- Create Enrollments table
CREATE TABLE Enrollments (
    enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER,
    course_id INTEGER,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

```
-- Insert a new student record
INSERT INTO Students (name, email, age) VALUES ('John Doe', 'john.doe@example.com', 20);

-- Fetch all courses enrolled by a student with student_id = 1 using JOIN
SELECT c.course_name, c.credits
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
WHERE e.student_id = 1;

-- Count the number of students in each course using GROUP BY
SELECT c.course_name, COUNT(e.student_id) AS student_count
FROM Courses c
LEFT JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_id, c.course_name;
```

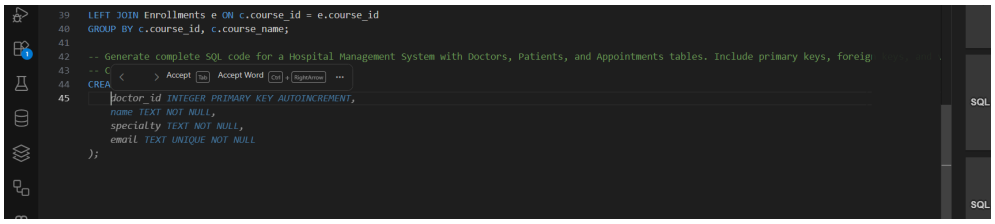


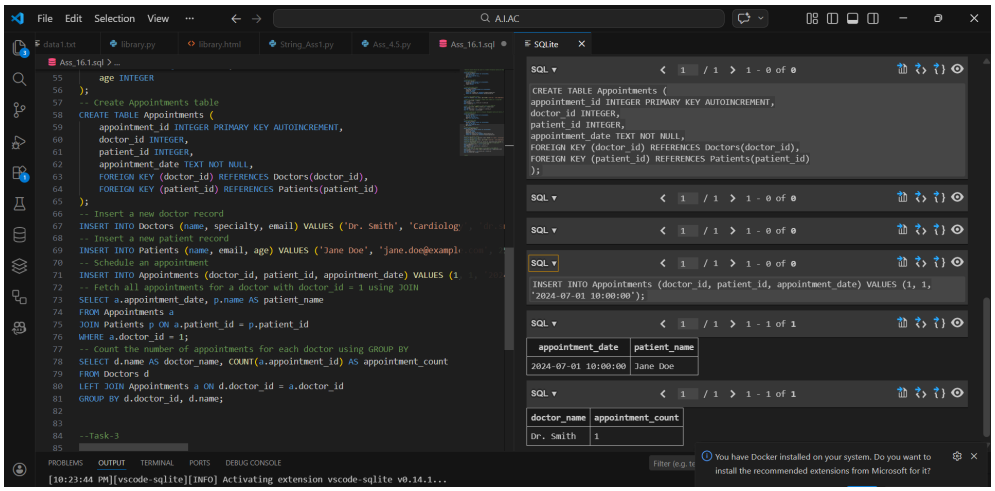


Task 2 – Hospital Management Database
Task:
Create schema and queries for a Hospital Management System.
Instructions:
• Tables: Doctors, Patients, Appointments.
• Use AI to define constraints (unique IDs, valid dates).
• Generate queries:
o List all appointments for a specific doctor.
o Retrieve patient history by patient ID.
o Count total patients treated by each doctor.
Expected Output:
• Normalized schema and SQL queries with joins.

```
--Task-2
-- Generate complete SQL code for a Hospital Management System with Doctors, Patients, an
d Appointments tables. Include primary keys, foreign keys, and various queries.
-- Create Doctors table
CREATE TABLE Doctors (
    doctor_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    specialty TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL
);
-- Create Patients table
CREATE TABLE Patients (
    patient_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    age INTEGER
);
-- Create Appointments table
CREATE TABLE Appointments (
    appointment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    doctor_id INTEGER,
    patient_id INTEGER,
    appointment_date TEXT NOT NULL,
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
);
-- Insert a new doctor record
INSERT INTO Doctors (name, specialty, email) VALUES ('Dr. Smith', 'Cardiology', 'dr.smith
@example.com');
-- Insert a new patient record
INSERT INTO Patients (name, email, age) VALUES ('Jane Doe', 'jane.doe@example.com', 25);
-- Schedule an appointment
INSERT INTO Appointments (doctor_id, patient_id, appointment_date) VALUES (1, 1, '2024-07
-01 10:00:00');
-- Fetch all appointments for a doctor with doctor_id = 1 using JOIN
SELECT a.appointment_date, p.name AS patient_name
```

```
FROM Appointments a
JOIN Patients p ON a.patient_id = p.patient_id
WHERE a.doctor_id = 1;
-- Count the number of appointments for each doctor using GROUP BY
SELECT d.name AS doctor_name, COUNT(a.appointment_id) AS appointment_count
FROM Doctors d
LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.doctor_id, d.name;
```



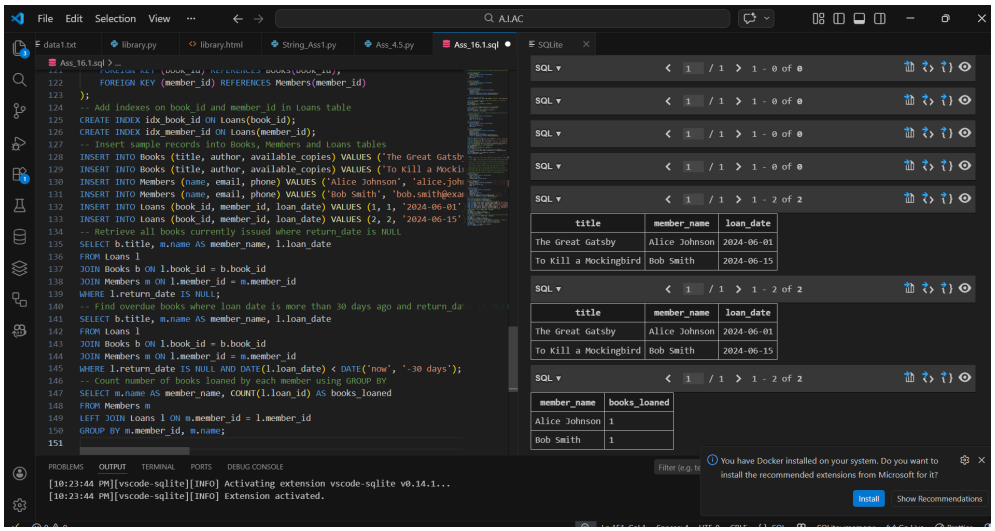


💡 Task 3 – Library Database
Task:
Design schema for a Library Management System.
Instructions:
• Tables: Books, Members, Loans.
• Use AI to suggest indexing strategy for faster queries.
• Generate queries:
o Retrieve all books currently issued.
o Find overdue books (loan date > 30 days).
o Count number of books loaned by each member.
Expected Output:
• Schema + SQL queries demonstrating joins and conditions.

```
--Task-3
-- Create a Books table with book_id as primary key (autoincrement), title, author, and a
vailable_copies fields in SQLite
```

```
-- Create a Members table with member_id as primary key (autoincrement), name, email and
phone fields in SQLite

-- Create a Loans table with loan_id as primary key, book_id and member_id as foreign key
s, loan_date and return_date fields in SQLite

-- Add indexes on book_id and member_id in Loans table for faster queries in SQLite

-- Insert sample records into Books, Members and Loans tables in SQLite

-- Retrieve all books currently issued where return_date is NULL in SQLite

-- Find overdue books where loan date is more than 30 days ago and return_date is NULL in
SQLite

-- Count number of books loaned by each member using GROUP BY in SQLite
-- Create Books table
CREATE TABLE Books (
    book_id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    author TEXT NOT NULL,
    available_copies INTEGER NOT NULL
);
-- Create Members table
CREATE TABLE Members (
    member_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    phone TEXT
);
-- Create Loans table
CREATE TABLE Loans (
    loan_id INTEGER PRIMARY KEY AUTOINCREMENT,
    book_id INTEGER,
    member_id INTEGER,
    loan_date TEXT NOT NULL,
    return_date TEXT,
    FOREIGN KEY (book_id) REFERENCES Books(book_id),
    FOREIGN KEY (member_id) REFERENCES Members(member_id)
);
-- Add indexes on book_id and member_id in Loans table
CREATE INDEX idx_book_id ON Loans(book_id);
CREATE INDEX idx_member_id ON Loans(member_id);
-- Insert sample records into Books, Members and Loans tables
INSERT INTO Books (title, author, available_copies) VALUES ('The Great Gatsby', 'F. Scott
Fitzgerald', 5);
INSERT INTO Books (title, author, available_copies) VALUES ('To Kill a Mockingbird', 'Har
per Lee', 3);
INSERT INTO Members (name, email, phone) VALUES ('Alice Johnson', 'alice.johnson@example.
com', '123-456-7890');
INSERT INTO Members (name, email, phone) VALUES ('Bob Smith', 'bob.smith@example.com', '0
```

```
98-765-4321');
INSERT INTO Loans (book_id, member_id, loan_date) VALUES (1, 1, '2024-06-01');
INSERT INTO Loans (book_id, member_id, loan_date) VALUES (2, 2, '2024-06-15');
-- Retrieve all books currently issued where return_date is NULL
SELECT b.title, m.name AS member_name, l.loan_date
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL;
-- Find overdue books where loan date is more than 30 days ago and return_date is NULL
SELECT b.title, m.name AS member_name, l.loan_date
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL AND DATE(l.loan_date) < DATE('now', '-30 days');
-- Count number of books loaned by each member using GROUP BY
SELECT m.name AS member_name, COUNT(l.loan_id) AS books_loaned
FROM Members m
LEFT JOIN Loans l ON m.member_id = l.member_id
GROUP BY m.member_id, m.name;
```

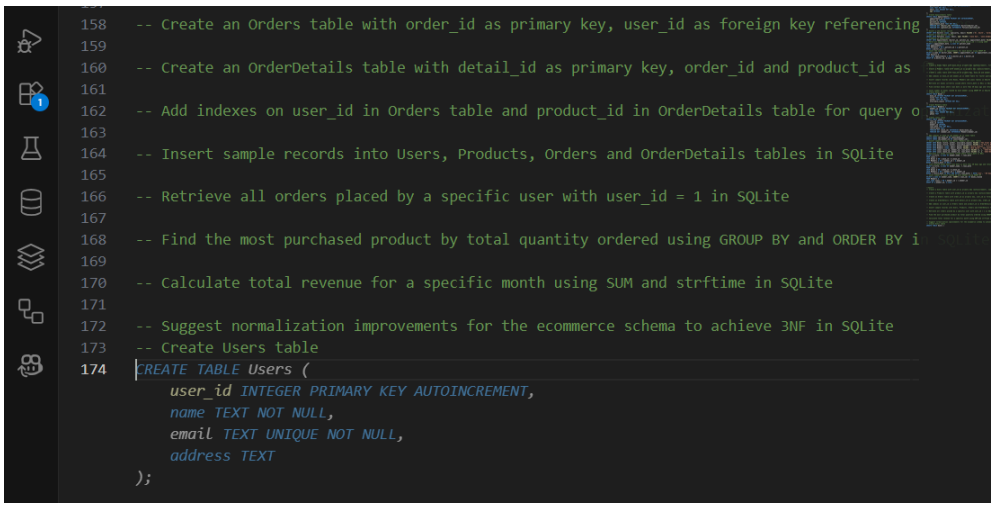


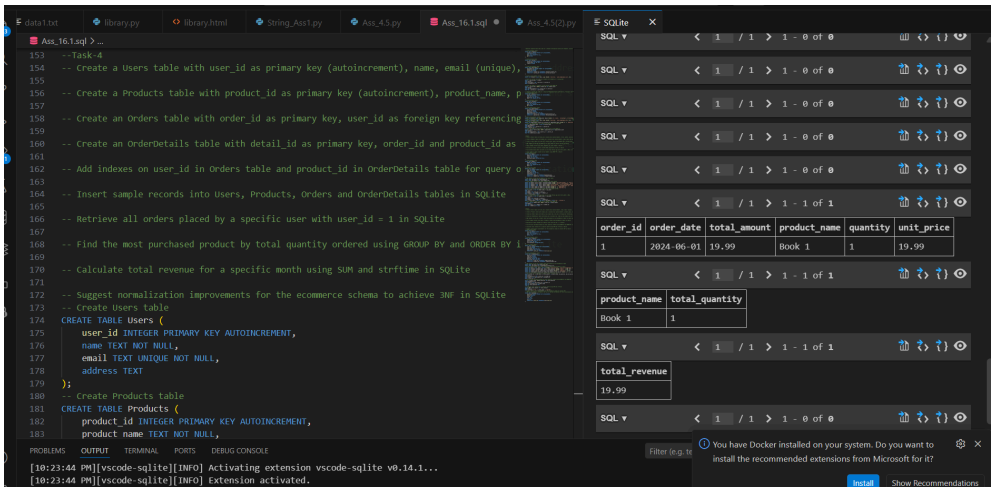
Task 4 – Real-Time Application: Online Shopping Database
Scenario:
Design a database for an E-commerce platform.
Requirements:
• Tables: Users, Products, Orders, OrderDetails.
• Generate queries to:
o Retrieve all orders by a user.
o Find the most purchased product.
o Calculate total revenue in a given month.
• AI should also suggest normalization improvements and query optimization.
Expected Output:
• Complete schema with relationships + SQL queries for analytics.

```
--Task-4
-- Create a Users table with user_id as primary key (autoincrement), name, email (uniqu
e), and address fields in SQLite

-- Create a Products table with product_id as primary key (autoincrement), product_name,
price and stock_quantity fields in SQLite

-- Create an Orders table with order_id as primary key, user_id as foreign key referencin
g Users, order_date and total_amount fields in SQLite

-- Create an OrderDetails table with detail_id as primary key, order_id and product_id as
foreign keys, quantity and unit_price fields in SQLite

-- Add indexes on user_id in Orders table and product_id in OrderDetails table for query
optimization in SQLite

-- Insert sample records into Users, Products, Orders and OrderDetails tables in SQLite

-- Retrieve all orders placed by a specific user with user_id = 1 in SQLite

-- Find the most purchased product by total quantity ordered using GROUP BY and ORDER BY
in SQLite

-- Calculate total revenue for a specific month using SUM and strftime in SQLite

-- Suggest normalization improvements for the ecommerce schema to achieve 3NF in SQLite
-- Create Users table
CREATE TABLE Users (
    user_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    address TEXT
);
-- Create Products table
```

```
CREATE TABLE Products (
    product_id INTEGER PRIMARY KEY AUTOINCREMENT,
    product_name TEXT NOT NULL,
    price REAL NOT NULL,
    stock_quantity INTEGER NOT NULL
);
-- Create Orders table
CREATE TABLE Orders (
    order_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER,
    order_date TEXT NOT NULL,
    total_amount REAL NOT NULL,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);
-- Create OrderDetails table
CREATE TABLE OrderDetails (
    detail_id INTEGER PRIMARY KEY AUTOINCREMENT,
    order_id INTEGER,
    product_id INTEGER,
    quantity INTEGER NOT NULL,
    unit_price REAL NOT NULL,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
-- Add indexes on user_id in Orders table and product_id in OrderDetails table
CREATE INDEX idx_user_id ON Orders(user_id);
CREATE INDEX idx_product_id ON OrderDetails(product_id);
-- Insert sample records into Users, Products, Orders and OrderDetails tables
INSERT INTO Users (name, email, address) VALUES ('Charlie Brown', 'charlie.brown@example.
com', '123 Main St');
INSERT INTO Products (product_name, price, stock_quantity) VALUES ('Book 1', 19.99, 10);
INSERT INTO Products (product_name, price, stock_quantity) VALUES ('Book 2', 29.99, 5);
INSERT INTO Orders (user_id, order_date, total_amount) VALUES (1, '2024-06-01', 19.99);
INSERT INTO OrderDetails (order_id, product_id, quantity, unit_price) VALUES (1, 1, 1, 1
9.99);
-- Retrieve all orders placed by a specific user with user_id = 1
SELECT o.order_id, o.order_date, o.total_amount, p.product_name, od.quantity, od.unit_pri
ce
FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id
JOIN Products p ON od.product_id = p.product_id
WHERE o.user_id = 1;
-- Find the most purchased product by total quantity ordered using GROUP BY and ORDER BY
SELECT p.product_name, SUM(od.quantity) AS total_quantity
FROM OrderDetails od
JOIN Products p ON od.product_id = p.product_id
GROUP BY od.product_id, p.product_name
ORDER BY total_quantity DESC
LIMIT 1;
-- Calculate total revenue for a specific month using SUM and strftime
SELECT SUM(o.total_amount) AS total_revenue
```

```
FROM Orders o
WHERE strftime('%Y-%m', o.order_date) = '2024-06';
-- Suggest normalization improvements for the ecommerce schema to achieve 3NF
-- The current schema is already in 3NF as all non-key attributes are fully functionally
dependent on the primary key and there are no transitive dependencies. However, if we wan
t to further normalize, we could consider separating the address field in the Users table
into a separate Addresses table to allow for multiple addresses per user. This would invo
lve creating an Addresses table with address_id as primary key, user_id as foreign key re
ferencing Users, and address fields (street, city, state, zip).
CREATE TABLE Addresses (
    address_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER,
    street TEXT,
    city TEXT,
    state TEXT,
    zip TEXT,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);
```

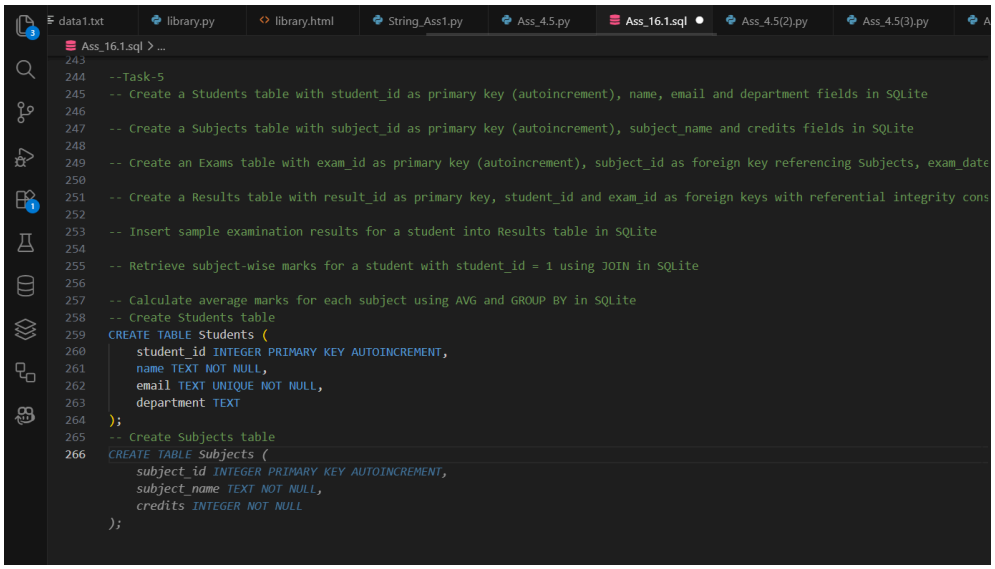


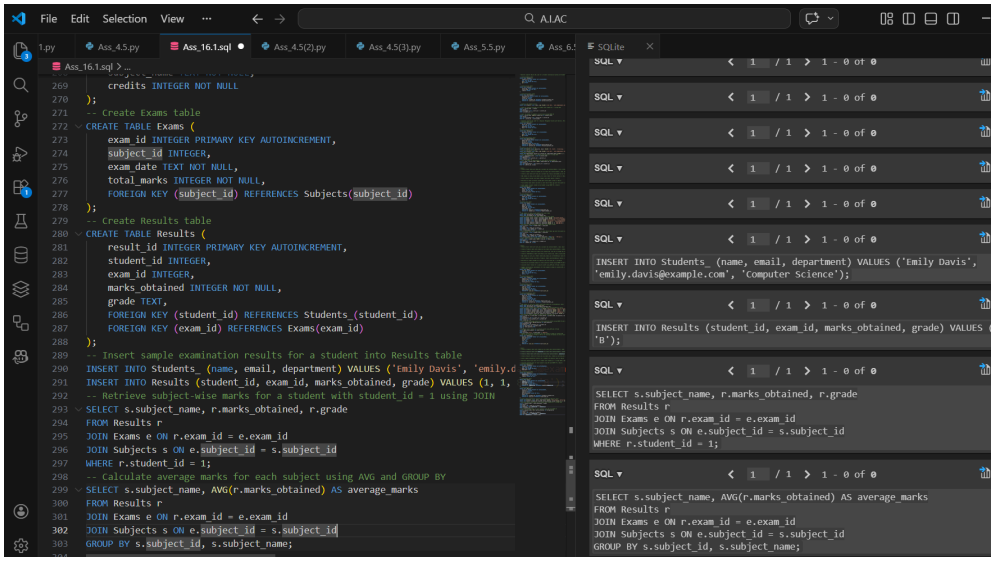


Task 5 – University Examination Database
Design a database schema for a University Examination System and generate SQL queries using AI.
Instructions:
• Tables: Students, Subjects, Exams, Results.
• Define primary keys, foreign keys, and referential integrity constraints.
• Generate queries:
o Insert examination results for a student.
o Retrieve subject-wise marks for a student.
o Calculate average marks for each subject.
Expected Output:
• Normalized schema and SQL queries involving aggregation and joins.

```
--Task-5
-- Create a Students table with student_id as primary key (autoincrement), name, email an
d department fields in SQLite

-- Create a Subjects table with subject_id as primary key (autoincrement), subject_name a
nd credits fields in SQLite

-- Create an Exams table with exam_id as primary key (autoincrement), subject_id as forei
gn key referencing Subjects, exam_date and total_marks fields in SQLite

-- Create a Results table with result_id as primary key, student_id and exam_id as foreig
n keys with referential integrity constraints, marks_obtained and grade fields in SQLite

-- Insert sample examination results for a student into Results table in SQLite

-- Retrieve subject-wise marks for a student with student_id = 1 using JOIN in SQLite

-- Calculate average marks for each subject using AVG and GROUP BY in SQLite
-- Create Students table
CREATE TABLE Students_ (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    department TEXT
);
-- Create Subjects table
CREATE TABLE Subjects (
    subject_id INTEGER PRIMARY KEY AUTOINCREMENT,
    subject_name TEXT NOT NULL,
    credits INTEGER NOT NULL
);
-- Create Exams table
CREATE TABLE Exams (
    exam_id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    subject_id INTEGER,
    exam_date TEXT NOT NULL,
    total_marks INTEGER NOT NULL,
    FOREIGN KEY (subject_id) REFERENCES Subjects(subject_id)
);
-- Create Results table
CREATE TABLE Results (
    result_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER,
    exam_id INTEGER,
    marks_obtained INTEGER NOT NULL,
    grade TEXT,
    FOREIGN KEY (student_id) REFERENCES Students_(student_id),
    FOREIGN KEY (exam_id) REFERENCES Exams(exam_id)
);
-- Insert sample examination results for a student into Results table
INSERT INTO Students_ (name, email, department) VALUES ('Emily Davis', 'emily.davis@examp
le.com', 'Computer Science');
INSERT INTO Results (student_id, exam_id, marks_obtained, grade) VALUES (1, 1, 85, 'B');
-- Retrieve subject-wise marks for a student with student_id = 1 using JOIN
SELECT s.subject_name, r.marks_obtained, r.grade
FROM Results r
JOIN Exams e ON r.exam_id = e.exam_id
JOIN Subjects s ON e.subject_id = s.subject_id
WHERE r.student_id = 1;
-- Calculate average marks for each subject using AVG and GROUP BY
SELECT s.subject_name, AVG(r.marks_obtained) AS average_marks
FROM Results r
JOIN Exams e ON r.exam_id = e.exam_id
JOIN Subjects s ON e.subject_id = s.subject_id
GROUP BY s.subject_id, s.subject_name;
```




File Edit Selection View
A.I.AC
1.py
Ass_4.5.py
Ass_16.1.sql
Ass_4.5(2).py
Ass_4.5(3).py
Ass_5.5.py
SQLite
Ass_16.1.sql > ...
credits INTEGER NOT NULL
);
-- Create Exams table
CREATE TABLE Exams (
exam_id INTEGER PRIMARY KEY AUTOINCREMENT,
subject_id INTEGER,
exam_date TEXT NOT NULL,
total_marks INTEGER NOT NULL,
FOREIGN KEY (subject_id) REFERENCES Subjects(subject_id)
);
-- Create Results table
CREATE TABLE Results (
result_id INTEGER PRIMARY KEY AUTOINCREMENT,
student_id INTEGER,
exam_id INTEGER,
marks_obtained INTEGER NOT NULL,
grade TEXT,
FOREIGN KEY (student_id) REFERENCES Students_(student_id),
FOREIGN KEY (exam_id) REFERENCES Exams(exam_id)
);
-- Insert sample examination results for a student into Results table
INSERT INTO Students_ (name, email, department) VALUES ('Emily Davis', 'emily.d
INSERT INTO Results (student_id, exam_id, marks_obtained, grade) VALUES (1, 1,
-- Retrieve subject-wise marks for a student with student_id = 1 using JOIN
SELECT s.subject_name, r.marks_obtained, r.grade
FROM Results r
JOIN Exams e ON r.exam_id = e.exam_id
JOIN Subjects s ON e.subject_id = s.subject_id
WHERE r.student_id = 1;
-- Calculate average marks for each subject using AVG and GROUP BY
SELECT s.subject_name, AVG(r.marks_obtained) AS average_marks
FROM Results r
JOIN Exams e ON r.exam_id = e.exam_id
JOIN Subjects s ON e.subject_id = s.subject_id
GROUP BY s.subject_id, s.subject_name;
SQL
1 / 1
1 - 0 of 0
INSERT INTO Students_ (name, email, department) VALUES ('Emily Davis',
'emily.davis@example.com', 'Computer Science');
INSERT INTO Results (student_id, exam_id, marks_obtained, grade) VALUES
'B');
SELECT s.subject_name, r.marks_obtained, r.grade
FROM Results r
JOIN Exams e ON r.exam_id = e.exam_id
JOIN Subjects s ON e.subject_id = s.subject_id
WHERE r.student_id = 1;
SELECT s.subject_name, AVG(r.marks_obtained) AS average_marks
FROM Results r
JOIN Exams e ON r.exam_id = e.exam_id
JOIN Subjects s ON e.subject_id = s.subject_id
GROUP BY s.subject_id, s.subject_name;